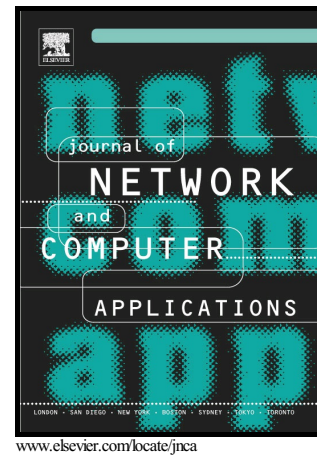


Ensemble based Collaborative and Distributed
Intrusion Detection Systems: A Survey

Gianluigi Folino, Pietro Sabatino



PII: S1084-8045(16)30024-8
DOI: <http://dx.doi.org/10.1016/j.jnca.2016.03.011>
Reference: YJNCA1601

To appear in: *Journal of Network and Computer Applications*

Received date: 22 August 2015
Revised date: 16 March 2016
Accepted date: 17 March 2016

Cite this article as: Gianluigi Folino and Pietro Sabatino, Ensemble based Collaborative and Distributed Intrusion Detection Systems: A Survey, *Journal of Network and Computer Applications*, <http://dx.doi.org/10.1016/j.jnca.2016.03.011>

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting galley proof before it is published in its final citable form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Ensemble based Collaborative and Distributed Intrusion Detection Systems: A Survey

Gianluigi Folino and Pietro Sabatino

*ICAR-CNR Istituto di Calcolo e Reti ad Alte Prestazioni
Via P. Bucci, 87036 Rende (CS)–Italy*

Abstract

Modern network intrusion detection systems must be able to handle large and fast changing data, often also taking into account real-time requirements. Ensemble-based data mining algorithms and their distributed implementations are a promising approach to these issues.

Therefore, this work presents the current state of the art of the ensemble-based methods used in modern intrusion detection systems, with a particular attention to distributed approaches and implementations. This review also consider supervised and unsupervised data mining algorithms, more suitable to work in an environment that requires the analysis of data streams in real-time. Sharing knowledge across multiple nodes is another of the key points in designing appropriate NIDS systems and for this reason, collaborative IDS were also included in this work. Finally, we discuss some open issues and lessons learned from this review, which can help researchers to design more efficient NIDSs.

Contents

1	Introduction	3
1.1	A panoramic on surveys in IDSs	4
1.2	Overview of the paper	5
2	Ensemble-based techniques	5
2.1	Supervised Data Mining	6
2.2	Unsupervised Data Mining	9
2.3	Hybrid data mining techniques	10
2.4	Statistical approaches and other techniques	11
2.4.1	Hidden Markov Models	11
2.4.2	Other statistical techniques	12
3	Distributed and collaborative IDS	13
3.1	Collaborative IDS	14
3.2	Data mining-based approaches	17
3.3	High-performance implementations	19
4	Discussion: Lesson Learned and Open Issues	20
5	Conclusions	22

1. Introduction

Network Intrusion Detection Systems (NIDS) are usually distinguished in *signature-based* or *misuse-based* intrusion detection systems and *anomaly-based* intrusion detection systems. In the misuse based approach, that can be summarized as “detect what I know”, when an attack has been discovered, the different steps of the attack are encoded in a *signature*, then the signature is stored in a database and finally it is used by the system to detect the attacks. On the contrary, anomaly-based systems, first try to model the normal behavior of the system to be protected, and then generate an alarm when a significant deviation from this normal behavior is detected. However, it is problematic to define opportunely how significant the deviation is in these kinds of systems, as it is necessary to find a trade-off between generating a large number of false alarms and letting some attacks elude the system.

As a consequence of our interconnected society and also due to the increased speed of the underlining network connections (gigabits per second) [65], computer network activities, human actions, systems, etc. generate large amounts of data and network traffic (typical examples of these streaming data in this particular domain are network logs, credit card transactional flows, and sensor network data). Indeed, this aspect must be seriously taken into account in designing modern NIDSs, which have to handle these large and fast-changing data. Typically, data mining-based algorithms are among the most used to this aim [34]. However, the ever-changing nature, the high volume of these data, the large amount of storage necessary, etc. can put in crisis traditional sequential data mining algorithms, as they are not able to capture new trends in the stream. In fact, traditional algorithms assume that data is static, i.e. a concept, represented by a set of features, which does not change because of modifications of the external environment.

A more flexible approach is that of ensemble-based algorithms [64], which have been stimulating an increasing interest in the intrusion detection community as they have some characteristics of interest in this particular domain. Indeed, they can be easily implemented in parallel/distributed architectures, improve the accuracy of a weak learner, can be specialized for the detection of one particular class and are particularly suitable to the case of unbalanced datasets.

In the last few years, distributed computing has considerably changed due to the emergence of new environments, such as grid and cloud computing. In particular, the Cloud Computing paradigm has enormous potential for intrusion detection systems; in fact, the availability of distributed resources on demand permits data and computational power to be accessed, previously hardly obtainable, and make it possible to deal with complex problems with real-time constraints. Furthermore, even for industries or institutions that do not have the opportunities to use expensive clusters, it is possible to design and to execute massively parallel algorithms also by exploiting the potentialities of the modern multicore CPU and GPU-based architectures.

The increase in the volume of data that need to be processed tends to push the computational load that the underlying hardware needs to sustain to its limits. Indeed, recall that in order to be really useful a NIDS should work in real time, and it needs to sustain the analysis of the data stream involved without saturating all the resources available.

Moreover, increased network connections also mean an increase in the number of alarms generated, especially in the case of signature-based NIDS. As a consequence, there is an ever increasing need for solutions that do not overwhelm human operators with a number of unmanageable alarms. Finally, NIDSs need to be capable of dealing with new undocumented attacks, for which a signature is not already available, i.e., the 0-day attacks.

In view of the previous considerations, this paper mainly reviews the ensemble-based algorithms based on data mining techniques that can be implemented in a parallel/distributed environment in as straightforward way as possible. Choice of reviewing mainly ensemble-based methods is due to the fact that they are not sufficiently treated in the literature for the task of intrusion detection.

More in detail, this review aims not only to analyze supervised data mining algorithms, but also works based on anomaly detection and, in particular on clustering suitable to operate in an environment that requires the analysis of data streams in real-time. Sharing knowledge across multiple nodes is another of the key points in designing appropriate NIDS systems and for this reason, collaborative IDS were also included in this work.

1.1. A panoramic of surveys on Intrusion Detection Systems

In this section, we review the main surveys in the field of intrusion detection and remark on the differences from our work.

Many surveys give a general overview of network anomaly detection systems, for instance [81] and [57]; an interesting survey of the field of anomaly-based network intrusion detection can be found in [44], which also includes a panoramic of the principal anomaly-based intrusion detection techniques, namely statistical-based, machine learning-based and knowledge-based. For each of the techniques illustrated, the main systems present in the literature are described. Moreover, the main challenges of the field are outlined and discussed.

However, one of the most comprehensive introductions to the field of *Network Anomaly Detection* (NAD) is presented in [10]. The main aim of this paper is to provide enough informations for a new researcher to acquire a certain familiarity with every aspect of the field. Accordingly, it contains a clear categorization of most common attacks encountered and of the main systems for network intrusion detection, according to the anomaly detection method and to the computational technique adopted. Based on this classification, an in-depth comparison is performed of the different architectures. A review of the principal tools that are useful to researchers in the network anomaly detection field is also included, as well as a comparative description of the datasets of network traffic publicly available.

Other surveys, described in the following, are more specific to a topic, types of data or technique. The problem of preprocessing and feature selection in the anomaly detection field is illustrated in detail in [32]. The review covers both automated methods for features extraction and techniques used to reduce the dimensionality of the problem. In fact, the authors analyze the literature on this topic and identify a shift from techniques that rely on the analysis performed by human domain experts to automatic solutions that implement a broad range of techniques of analysis.

Generally, systems are categorized according to the detection paradigm considered and to the type of attack, i.e., denial of service, worms, etc. The work in [93] concentrates on the intrusion detection systems that operate at the level of network flow. In [77], the authors present a review of the main machine learning techniques employed in the network intrusion detection field. The techniques considered include rule-based learning, decision tree, Bayesian reasoning, neural networks, support vector machines and clustering. Moreover, a section devoted to nature-inspired techniques is also included, which covers artificial immune systems, genetic programming and swarm intelligence. In addition, a comparison of the performance of the different techniques is illustrated, reporting experimental results contained in the literature. Finally, the current challenges of the field are discussed.

In [46], a panoramic of tools useful to researchers in the intrusion detection field is presented. In particular, the main steps necessary to perform an attack are described. Scanning and sniffing tools are discussed, as well as an extensive discussion of attack launching tools. Network monitoring systems and working attack detection systems are also covered. Finally, the main challenges that researchers in the field have to face are the subject of the closing considerations of the work. The same authors also reviewed the topics of port scans, scanning tools and launching attacks in [12] and the main detection techniques in [9, 11].

More correlated to our paper, two surveys of network intrusion detection systems from the distributed point of view can be found in [109] and [95]. Zhou et al. [109] point their attention to coordinated attacks such as large-scale scans, worm diffusion and distributed denial of service and on the research in the field of collaborative intrusion detection that aims to detect and prevent these kinds of attack. The work describes the different architectures of a collaborative intrusion detection system and also illustrates some algorithms used for the task of alert correlation. Vasilomanolakis et al., [95] outline the general architecture of a collaborative intrusion detection system. The main building blocks of these systems and the methodology to integrate them in a common environment are described. In comparison with the latter two papers, our work is more focused on the ensemble-based paradigm, and on unsupervised and supervised data mining techniques. In addition, we analyzed the main distributed algorithms used for these topics, while the above-cited papers are more aimed at a general discussion of the possible collaborative and distributed architectures, without discussing the interesting ensemble-based techniques.

Another related work [25] reviews many state-of-the-art solutions for computer security from the point of view of the information fusion and proposes a general scheme to perform information fusion for intrusion detection in computer systems. In addition, the work discusses strengths and weaknesses of currently used approaches and examines some interesting research issues. However, the paper is less specific than our, as it treats computer security in general, and furthermore it does not analyze parallel/distributed solutions.

Indeed, to the best of our knowledge, there are no up to date surveys in the literature on distributed and collaborative systems with particular emphasis on the ensemble approach.

1.2. Overview of the paper

The paper is organized as follows: Section 2 surveys NIDS systems based on the ensemble techniques, mainly analyzing data mining approaches: supervised, unsupervised and also hybrid solutions; in addition a subsection is devoted to discussing ensemble systems based on other techniques, i.e., Hidden Markov models; Section 3 provides a description of the distributed and collaborative approaches, in particular the main works concerning collaborative IDS, data mining distributed systems and high-performance implementation are illustrated; Section 4 discusses the strengths and weaknesses of the reviewed approaches and some open issues in the field; finally, Section 5 concludes the paper.

2. Ensemble-based techniques

Ensemble-based techniques permit one to combine multiple (heterogeneous or homogeneous) models in order to classify new unseen instances. In practice, after a number of classifiers are built usually using some parts of the original dataset, the predictions of the different classifiers are combined and a common decision is taken. Different schemas can be considered to generate the classifiers and to combine the ensemble, i.e. the same learning algorithm can be trained on different datasets or/and different algorithms can be trained on the same dataset.

The best known and used ensemble-based algorithm is the boosting algorithm, first introduced by Schapire [85] and Freund [86]. In order to boost the performance of any “weak” learning algorithm, i.e. an algorithm that “generates classifiers which need only be a little bit better than random guessing” [86], the boosting method adaptively changes the distribution of the training set according to how difficult each example is to classify. This algorithm, in common with many ensemble-based algorithms, has the drawback of needing to repeat the training phase for a number of rounds and that is really time-consuming in the case of large datasets or not practicable in the case of real-time requirements, i.e. in the intrusion detection domain. On the contrary, other variants of ensemble-based algorithms use functions to combine the classifiers that do not need to reuse the original training set. The majority vote is a classical example of this kind of combiner

function, but there are many other functions with this property and they are named non-trainable combiners [64]. Combiner functions having this property considerably reduces the time requirements required to compute them, as it is not necessary to reuse the training set in this phase.

The information fusion approach, which combines information coming from different sources in order to build new aggregate features or to enhance the solution of a problem, may permit to solve better or more efficiently an IDS task [30]. In fact, different aspects can motivate the use of information fusion for this kind of problems, i.e., information may be present at multiple abstraction levels and may be collected from multiple sources, different abstraction levels can be present in the data, it is effective in increasing timeliness of attack identification and in reducing false alarm rates and usually obtain high detection rate, etc. However, as an additional computational overhead must be also taken into account to process these different sources, a fast and effective fusion of information is fundamental to enhance the value of the classification and clustering task and the computational cost of the process must be taken into account. Many fusion-based techniques can be applied to the intrusion detection task. An interesting classification [10] considers three cases depending whether the algorithm operates at the level of the data (data level), of the features (feature level), and of the final decision (decision level). Indeed, some methods try to consider the semantic groups in which the high dimensionality space of the features is divided or the hierarchical abstraction levels or the type of information contained. Furthermore, as the intrusion detection task can be computationally expensive, many of the information fusion-based approaches, which we review in the next section, adopt non-trainable functions to combine the classifiers.

IDSs benefit from using information fusion and ensemble-based algorithms for a number of reasons. First, these techniques perform well both when data are very scarce and when we have a huge amount of data; furthermore, they can be easily implemented in efficient computing environments such as parallel, multi-core, and GPGPU architectures and also on P2P and Cloud computing environments. In addition, they can easily model different abstractions or parts of a network, i.e., some models can be trained on some parts or on some levels of the network and finally combined together, to assure a better prediction.

More in detail, in the next subsections, we analyze different variants of ensemble and information fusion-based approaches for the intrusion detection task: supervised and unsupervised data mining approaches, hybrid techniques combining the previous two approaches and some interesting statistical techniques. A summary of the main characteristics (i.e., the technique adopted, the preprocessing phase used, the function combining the ensemble, the datasets on which the experiments are conducted, the metrics used for evaluation, etc.) of these works is given in Table 1 (data mining-based approaches) and in Table 2 (statistical approaches and other techniques).

2.1. Supervised Data Mining

The technique of n -grams has been extensively employed for different kinds of problems, i.e., the statistical analysis of texts, [66, 29], and the problem of speech recognition. A model of the text is usually obtained by a sliding window of n -characters that form an n -gram. At steps of one character, the n -grams captured constitutes a set that represents the text. The use of n -grams has a computational cost that grows exponentially in n , as the feature space grows as 256^n ; therefore, this technique is computationally expensive. Nonetheless in protocols like , the content of a request is encoded in plain text, hence the name text-based protocols, and from here it follows that the technique of n -grams is clearly suitable to model the packet payload containing data, subsequently analyzed in search of anomalous content.

The pioneer work on applying n -grams to intrusion detection is described in [100], in which the PAYL network intrusion detection system is presented, while extensions of this work are discussed in [98], [99].

In the paper [78], an IDS based on an ensemble of SVMs is proposed. In this approach, the payload is modeled using the technique of 2_v -grams. Namely, for a fixed v , the payload is represented as a sequence

of 2-grams extracted at intervals of v bytes. Varying the value of v , a representation of the payload in different feature spaces is obtained. It is worth noting that each feature space obtained in this way has dimension 256^2 . In order to reduce the dimension of the feature space, the authors adopt a preprocessing stage, i.e., a clustering algorithm based on the technique originally proposed in [33]. In the training phase, an ensemble of SVMs, one model for each representation of the payloads, is build in order to model the normal traffic. In the detection phase, whenever a packet is received, each representation is classified by the corresponding SVM, giving as output an estimate of the probability that the payload belongs to normal traffic. The output probabilities are then combined using a non-trainable combiner, chosen among the average, the product, the minimum or the maximum. The output is then an overall estimate of the probability P of the payload being normal. Finally, the payload is then classified as normal if P is above a prefixed threshold. Experimental results were conducted on the well-known DARPA dataset and on the GATECH dataset, obtained by collecting seven days of real HTTP requests towards the website of the College of Computing School at the Georgia Institute of Technology, considered as normal traffic; subsequently, synthetic attacks generated by the polymorphic shell-code engine CLET¹ were added to the dataset. The approach is compared to PAYL and it results more effective in detecting polymorphic attacks and in detecting attacks with a very low false positive rate. However, the system has execution times higher than PAYL. A prototype of the system, also comprising the source code is available for download ².

In [16], a number of classifier algorithms, i.e., ANN, SVM, the Decision Tree and k -NN are trained and tested on a subset of the DARPA dataset and compared using detection and false positive rates. After that, the above-mentioned classifiers are combined in an ensemble by means of a combination rule, chosen among the majority voting rule, the average rule and the “belief” function, i.e., a function based on the estimation of the probabilities that a pattern assigned to a given data class actually belongs to that class or to other classes. Experimental results show that the ensemble-based approach tends to provide more reliable results compared with the approach based on a single classifier. In particular, the “belief” combination rule obtains better results.

In [1], Aburomman et al. train two types of base classifiers, SVM-based and k -NN-based, on the KDD’99 dataset. After this preliminary stage, the base classifiers are combined in an ensemble by a weighted majority voting. Three techniques are employed in order to chose the weights of the combination function: one based on the particle swarm optimization (PSO), another based on a variant of the PSO, which uses local unimodal sampling in order to iteratively varying the search range of the parameters in an optimal way, and lastly the Weighted Majority Algorithm, introduced by Littlestone et al. [70]. Experimental results show that the ensemble technique is able to improve the results obtained from the base classifiers alone in term of classification accuracy. Moreover, the best results are obtained by using the combination function, which adopt the technique based on the variant of the PSO.

An ensemble of neural networks is the basis of the architecture proposed in [89]. In particular, the Adaboost algorithm is used to train an ensemble on the KDD’99 dataset. In order to select the optimal subset of the features from the dataset, a genetic algorithm is employed, in which the individuals in the population are ensembles trained on a specific subset of the features and the fitness function is the accuracy on a validation set. It is well known that models based on neural networks tend to involve a certain complexity. To avoid this drawback, a decision tree is built by running the C4.5 algorithm on the classes obtained from the above-described technique. Experimental results show that the overall solution is better than the solutions based on neural networks and decision trees alone, with respect to different metrics, i.e., True Positive and False Positive Rate, Precision, Recall and F-measure.

¹

²https://pralab.diee.unica.it/en/HMMPayL_and_McPAD

Author(s)	Year	Technique(s)	Ensemble	Dataset(s)	Preprocessing	Metric(s)
<i>Supervised</i>						
Perdisci et al. [78]	2009	SVM	Average, product, minimum and maximum rule	DARPA, GATECH (Real traffic and injected attacks)	Clustering based [33]	ROC Curve
Borji [16]	2007	ANN, SVM, Decision Tree, k -NN	Majority voting, average and "belief" function	DARPA 98	None	Detection Rate, False Positive Rate
Aburomman et al. [1]	2016	SVM, k -NN	Weighted majority voting chosen by PSO	KDD'99	None	Classification accuracy
Sivatha et al. [89]	2013	Decision Tree, C4.5	AdaBoost	KDD'99	Genetic Algorithm	True and False Positive Rate, Precision, Recall and F-measure
<i>Unsupervised</i>						
Zhang et al. [104]	2006	Random forest	Majority voting	KDD'99	None	ROC Curve
Song et al. [55], [91]	2009, 2013	Density based clustering, SVM	Maximum rule	Real traffic	Feature selection based on the results of Mukkamala et al. [73]	ROC Curve
Bhuyan et al. [13]	2012	Tree-based subspace clustering algorithm (TreeCLUS)	Majority voting, maximum rule	KDD'99, TUIDS	Information theoretic based	Detection Rate, False Positive Rate
Giacinto et al. [45]	2008	k -means, v -SVC, Parzen-window density estimation	Maximum, minimum, mean and product rule	KDD'99	None	ROC Curve
<i>Hybrid</i>						
Hornig et al. [47]	2011	SVM	None	KDD'99	Data mining based, BIRCH algorithm [106]	Accuracy, False Positive Rate
Nguyen et al. [76]	2011	k -means algorithm, Decision Tree	Weighted mean	KDD'99	None	True Positive Rate, False Positive Rate
Elabiony et al. [37]	2013	Weighted variant of K -means	Random Forest	KDD'99	None	ROC Curve
Singh et al. [90]	2015	OS-ELM	Feature aggregation stage	NSL-KDD, Real traffic	Filtered, consistency, CFS subset evaluation and DBSCAN	Accuracy, True and False Positive Rate, True and False Negative Rate, F1-score, Precision and Execution Time

Table 1: Summary table of papers reviewed, data mining techniques

2.2. Unsupervised Data Mining

The supervised IDSs, described in the previous subsection, need to be trained on labelled datasets containing normal traffic and most of them also require that a considerable number of attacks are present in the training set. In this phase, usually the statistical behavior of the normal traffic is established and the parameters of the model are tuned accordingly. In many cases, a labelled dataset is difficult to obtain or only a few labeled tuples are available, and therefore unsupervised data mining algorithms are more suitable to cope with this problem. In the following, we review the main papers concerning unsupervised algorithms based on the ensemble model used in the intrusion detection field.

An unsupervised IDS framework based on the random forest algorithm is described in [104]. The system comprises a preprocessing phase that analyzes the network traffic and generates a dataset. Then, an analysis is performed off-line, as the computational cost of the algorithm employed, i.e., random forest, make it unfeasible for on-line analysis. The random forest algorithm (see [19]), is made up of an ensemble of regression trees and it generates a different tree for each bootstrap sample of the dataset. When an object needs to be classified, each tree gives a vote and the overall class is chosen by performing a majority voting. After the training stage, a proximity function between the instances is established and, in the detection stage, a number of outliers are individuated, based on the fact that the distance exceeds a prefixed threshold. The approach is evaluated on the well-known KDD'99 dataset and experiments prove that its accuracy is comparable to previously reported approaches, but it achieves a higher detection rate maintaining the false positive rate relatively low.

Another unsupervised approach based on data mining techniques is described in [55] and in [91]. In the first paper, the overall architecture of an IDS is described, which consists mainly of three stages, i.e., filtering, clustering and modeling. During the training phase, the attacks are filtered and eliminated from the training data. This step is accomplished by an algorithm that employs a notion of density, which is determined by the ratio of attacks with respect to normal traffic; this parameter needs to be specified by the user. After the data has been filtered, the system performs a clustering of the training data. Again, this step is determined by a parameter specifying the number of clusters to be obtained. The clusters obtained represent different models of normal traffic. Finally, in the modeling stage, for each cluster obtained, a one-class SVM is trained. In the testing phase, the traffic instances are evaluated by the ensemble of SVMs obtained in the previous step. Instances are flagged as normal, if their representation falls inside one of the one-class SVMs. In [91], the authors extended the proposed method in order to tune the values of the above-mentioned parameters automatically without any intervention by the user. The architecture is evaluated on real traffic data obtained from Kyoto University. The two approaches are compared and the results show the superiority of the automatic approach, its efficiency in determining the suitable number of clusters and in estimating the ratio of attack instances with respect to normal traffic.

In [13], a fully unsupervised approach is presented based on a clustering technique. The clustering stage of the preprocessed data is performed by the TreeCLUS algorithm. This clustering technique is a tree-based subspace clustering algorithm that employs information theoretic concepts (see [4]) in order to identify the most relevant features. Moreover, TreeCLUS performs a cluster stability analysis by using an ensemble and each element of the ensemble uses a different clustering stability measure to perform the task. The measures employed are *Dunn index* [36], *C-index* [50], *Davies Bouldin index* [31], *Silhouette index* [83], *Xie-Beni index* [103]. The results obtained by the classifiers are combined by a simple majority voting. Finally, the clusters obtained by the TreeCLUS algorithm are labeled by the CLUSLab algorithm, which also employs an ensemble approach, described in the following. After computing the cluster size, the compactness and the dominating features subset, it labels each cluster as normal or anomalous by using a maximum decision score-based function. The authors test the two algorithms on the well-known KDD'99

and on the real-world TUIDS datasets, and conclude that both the approaches outperform other existing unsupervised network anomaly detection techniques for the metrics of detection rate and false positive rate.

The approach adopted by Giacinto et al. [45] is based on a modular ensemble. In practice, each classifier contained in the ensemble is specialized on the classification of the traffic concerning a specific service, e.g. web service, mail service and so on. Three solutions, all density-based, for the base classifiers are tested: a Parzen-window approach [35], an approach based on the K -means clustering algorithm and ν -SVC, a solution inspired to SVM, see Schölkopf et al. [87] for more details. Then, the base classifiers are combined in the ensemble by using simple non-trainable rules, i.e., maximum, minimum, mean and the product rule. Experiments on the KDD'99 dataset prove that the most efficient approach in terms of the area under the ROC curve is the one based on the ν -SVC technique.

2.3. Hybrid data mining techniques

In this section we collect techniques that combine the advantages of the supervised and unsupervised paradigms, or that include both a component based on misuse detection and on anomaly detection.

The approach [47] relies on the BIRCH clustering algorithm, see [106]. In particular the KDD'99 dataset is split in to its five principal classes, Probe, Dos, U2R, R2L and normal traffic. The above-mentioned clustering algorithm is employed to build five clustering feature trees, one for each class. Roughly speaking, the clustering feature tree is a compact representation of the dataset, in which each leaf node corresponds to a cluster. After this compact representation of the dataset is obtained, a feature selection is performed. Four SVMs, one for each attack class, are then trained and combined together in an ensemble for subsequent testing, performed again on the KDD'99 dataset. Compared to solutions employing different techniques, such as decision trees and K -means, it shows slightly better or comparable performances.

In the approach proposed by [76], data collected from heterogeneous sources such as network traffic packets, operation system audits, and system log files are preprocessed to form a single dataset. The dataset is labeled by means of domain expert knowledge, and other semi-automatic methods to constitute the IDS training set, which is split into a training and a validation set. Then, a clustering of the data is performed by using the K -means algorithm, which infers additional features. Subsequently, a decision tree algorithm is employed to build an ensemble of different classifiers. A weighted mean is used as combiner strategy for the ensemble, where the weights are selected according to the classification abilities of the single classifier established in the validation phase. Experiments are conducted on the benchmark dataset KDD'99 and they prove that the system is able to outperform different ensemble-based algorithms, i.e., bagging and boosting.

A hybrid approach to the intrusion detection problem is presented in [37]. In particular, misuse and anomaly-based approaches are combined in the following way. First of all, starting with a labeled dataset, the rules for detecting the attacks are generated by a random forest algorithm and then, these rules are employed in the misuse detection stage. Finally, the anomaly detection phase is accomplished by clustering a new training set using the K -means algorithm. Indeed, a weighted variant of the K -means algorithm is adopted, where the weights assigned to the features are extrapolated from the output of the random forest algorithm. In order to detect anomalous clusters, known attacks are injected in the training dataset and clusters containing a great number of injected attacks are labeled as anomalous. In the operational phase, the network traffic is first analyzed by the misuse detection stage and, if the traffic does not match any known attack, then it is passed to the anomaly detector that analyses it in order to discover a new kind of attacks. Experiments conducted on the KDD'99 dataset show that this system is able to outperform two other unsupervised anomaly detection frameworks, described in [38, 105]; it is worth noticing that the second is also based on a random forest algorithm.

An hybrid system based on neural networks is presented in Singh et al. [90]. The feature selection stage of the NIDS is based on an ensemble of three different techniques: filtered subset evaluation, consistency

subset evaluation and CFS (correlation feature selection) subset evaluation, see Witten et al. [102]. Moreover, during the training stage, in order to reduce considerably the number of samples to be processed and the overall memory consumption, duplicate and similar connections are clustered by the DBSCAN algorithm. In practice, the centers of these clusters represent profiles of connections. Finally, the classification stage is performed by a neural network based algorithm, OS-ELM (Online Sequential Extreme Learning Machine), trained on the connection profiles that result from the previous stage. Experimental testing is conducted on NSL-KDD and on real traffic collected from the Kyoto University. Different tests are conducted in order to assess that each stage of the proposed architecture improves the overall efficiency with respect to the following metrics: detection accuracy, true positive, true negative, false positive, false negative rate, F1-score, precision and execution time.

2.4. Statistical approaches and other techniques

In this section, we will direct our attention to works that are based on statistical models for the analysis of the incoming network traffic. Among these techniques, the most relevant are based on Hidden Markov Models (HMM). Basically HMMs are a Markov process like Markov chains, but contrary to Markov chains, the transition probability functions between the states is not set a priori but it is determined in the training phase. After considering works based on HMMs, we briefly describe a system that employs the Naïve Bayes Classifiers in the ensemble and one that exploits the use of Neyman-Pearson Hypothesis Testing in order to reduce the false positive detection rate.

2.4.1. Hidden Markov Models

In [6, 5], an IDS based on an ensemble of HMMs is presented. Differently from the 2_v -gram analysis of the payload used in the paper [78], discussed in the Subsection 2.1, this approach models sequences of length n , avoiding the problems due to the approximation of the analysis based on 2_v -grams. In practice, a window of n bytes slides over the payload and it is treated as a sequence that is inserted in the set that represents the payload. This selection produces a certain redundancy intuitively, and this is shown by a quantitative analysis in the paper. In order to alleviate this problem, only a subset of the sequences, selected randomly from the payload representation, is passed to the HMMs for further analysis; this approach has the obvious outcome of speeding up the IDS. Moreover, it provides an additional line of defense against attempts to evade the IDS, as the attacker has indeed no clue of which part of the payload is used for the analysis. The set of sequences obtained in the preprocessing phase is then passed to the classification stage of the IDS. In the training phase, the parameters of each one of the K HMMs are established in order to maximize the probability assigned by the HMM model to the packets of normal traffic in the training set. The difference in the K models is obtained by choosing different random initialization matrices. In the detection phase, each HMM produces a probability for the set of sequences extracted from the payload. Then, the K estimates are combined, similarly to the 2_v -gram approach of Perdisci et al., using non-trainable combiners, i.e. the minimum, the maximum, the mean, and the geometric mean and consequently an output probability P is assigned. Finally, the payload is classified as suspicious if P is above a predefined threshold. For the purpose of testing the system, the authors employ three datasets of normal traffic, namely the classic DARPA'99, a dataset containing the HTTP requests sent to the web site of the College of Computing of the University of Georgia Institute of Technology (both these two datasets are the same as employed in the evaluation of McPAD, (see 2.1) and a dataset containing the HTTP requests sent at website of Department of Electrical and Electronic Engineering of the University of Cagliari. In addition to the normal traffic datasets, a database of generic HTTP attacks ([51]) and variants of the generic attacks generated by a polymorphic engine CLET (the same attacks used in the evaluation of McPAD) were employed. Moreover, simulate XSS-SQL attacks to the web server of the University of Cagliari were also added [24]. As for the accuracy, HMMPayl is able

to outperform PAYL and McPAD and in addition, it is particularly efficient in detecting attacks based on Cross-Site Scripting and SQL-Injection, in spite of these attacks present payload statistics not significantly different from the normal traffic. However, the processing time of HMMPayl is significantly higher than PAYL, even if the authors point out that the MCS approach can be easily parallelized. A prototype of the system complete of source code is available for download ³.

Although developed in a completely independent way, a similar approach to HHMPayl is described in [92]. The proposed IDS, named Spectrogram, works at the level of the packet payload. In particular, it analyzes HTTP requests in order to discard irrelevant content. Then, the payload is modeled by employing a technique based on N -grams, and, in particular, the system adopts a sliding window of prefixed width to inspect the payload. Analogously to HMMPayl, an ensemble of Hidden Markov Models is employed to model the sequence of N -grams extracted from the payloads. During the training phase, the hidden parameters of the set of HMMs are determined by machine learning techniques. The user has to choose only two parameters, namely the gram-size and the number of HMMs in the ensemble. Experimental results are conducted on real traffic captured from a couple of operational web servers with added the traffic relative to a number of well-known attacks generated using available exploits. The evaluation considers how the choice of the main parameters (i.e., the gram-size and the number of HMMs) influences the performance of the system and, in addition, some techniques to optimally select the two above-mentioned parameters are proposed. Finally, compared to a work based on N -grams, i.e. Anagram [99], the approach based on HMMs scale better with respect to the gram size, in spite of maintaining a reasonable computational load.

Although in this review we direct our attention to intrusion detection systems that rely on the inspection of the network traffic alone, however there are other interesting approaches, named host-based intrusion detection systems, which are worth mentioning. In these kinds of approaches, many aspects of the system are monitored, from the CPU load to the user inputs. Examples of this paradigm are the two works described in the following. In the system proposed in [60], HMMs are employed with the objective of generating models of normal activity for specific users; in practice, the models are linked to parameters such as CPU activity, systems call activity, system process activity, network activity, and session activity. Anomalous activities are then identified if they deviate from normal activities. In order to keep track of periodic changes in the user profiles, the system implements a concept drift detector based on the Kullback-Leibler divergence [61]. In [59], the same authors implement the system in a distributed environment, i.e. they consider a situation in which the data are collected from different systems in a subnetwork and then correlated in a central node. For further discussion on the topic of distributed intrusion detection systems, we refer to Section 3.

2.4.2. Other statistical techniques

A system based on data mining and statistical techniques is described in [17]. The authors pursue a meta-ensemble approach, namely a set of classifiers is combined by various simple non-trainable rules and finally different ensembles are combined using a weighted majority voting. First of all, in the preprocessing stage, nfdump is used to acquire the network flow. In the subsequent training phase, methods based on the concept of information gain, previously introduced in [56], are used in order to determine the most relevant feature for each class, into which the traffic is subdivided. The classification stage is then performed by three principal layers. The first layer consists of a set of basic classifiers, i.e., the C4.5 Algorithm, the Naïve Bayes Classifiers and the Decision Table. Each classifier assigns a value, representing the probability of belonging to a specific class, to each instance of the traffic flow; then, in the second layer, the probabilities are

³https://pralab.diee.unica.it/en/HMMPayl_and_McPAD

combined using different functions: i.e., sum, mean, minimum, maximum, median and weighted average. The third layer combines estimates of the previous layer by using the weighted majority voting rule, where basically a weight is assigned to a particular rule based on its past performance. The method proposed is compared to Adaboost.M1, Bagging and Majority Voting and exhibits better performance for all the cases. Datasets used for the comparison comprehend the well-known KDD'99 and two additional datasets, part of the Tezpur University intrusion detection system (TUIDS) datasets. These datasets were generated by the authors deploying a subnetwork and launching attacks against it using exploits available at Packet Storm ⁴.

The problem of detecting DDoS attacks is faced in [63]. The solution proposed is based on an ensemble of supervised Adaptive Neuro-Fuzzy Systems (ANFS), see [53]. These base classifiers are combined in an ensemble by a variation of the AdaBoost algorithm. Feature extraction and selection is performed at the traffic flow level, extracting six features from traffic dump, among the most relevant with respect to the type of attacks considered. The algorithm, moreover, incorporates a mechanism to reduce the false positive detection based on Neyman-Pearson Hypothesis Testing, [88]. Experiments are conducted both on publicly available datasets (DARPA and CAIDA⁵) and on real traffic data captured from the servers at the University of Naples "Federico II". In the experiments, the system is able to outperform analogous solutions based on ensemble techniques as bagging and boosting. A similar approach is presented in [62]. The main difference with respect to the above system is the choice of resilient back propagation (RBP), [82], as the base classifier for the ensemble.

Author(s)	Year	Technique(s)	Ensemble	Dataset(s)	Preprocessing	Metric(s)
Ariu et al. [6]	2011	HMM	Minimum, maximum, mean and geometric mean	DARPA, Real traffic	Random projection	ROC Curve
Song et al. [92]	2009	HMM	Weighted sum	Real traffic	None	ROC Curve
Khanna et al. [60, 59]	2006, 2008	HMM	Weighted sum	DARPA, simulated subnetwork	None	ROC Curve
Boro et al. [17]	2012	C4.5, Naïve Bayes, and Decision Table	Meta ensemble, weighted majority vote	KDD'99, TUIDS	Information Gain Based	Classification Accuracy
Kumar et al. [63, 62]	2011, 2013	Adaptive Neuro-Fuzzy Systems (ANFS), see [53], Resilient Back Propagation (RBP), [82]	Modification of AdaBoost	DARPA, CAIDA, Real traffic	Based on classification accuracy	Detection Accuracy

Table 2: Summary table of papers reviewed, statistical approaches

3. Distributed and collaborative IDS

Attacks conducted against online systems which occur in multiple networks simultaneously (i.e. large-scale stealthy scans, worm outbreaks and distributed DDoS), are among the most dangerous. In addition, coordinated attacks are really difficult to detect using isolated intrusion detection systems since these systems usually monitor only a small portion of a network. Moreover, it is often the case that a particular IDS is more efficient in detecting certain type of attacks, while a different one is more suitable for other type of attacks. For instance, it is well known that misuse based IDSs are very efficient in detecting known attacks based on a database of signatures, but they are rather weak in detecting new attacks. On the contrary, anomaly based IDSs are able to detect a certain number of new attacks but they usually show a rather high false positive rate even in the case of known attacks. As already remarked, architectures of IDSs based

⁴

⁵

on the anomaly detection paradigm implement relatively complex analysis techniques in order to detect outliers in the network traffic being monitored. For instance, these techniques can be either data mining based or statistical based, but in any case, in order to attain a suitable detection rate and to minimize the false positive rate, they usually involve a rather high computational complexity. As a consequence, in order to cope with the high volume data involved in a real time environment, the natural choice is to exploit the potentiality of modern parallel hardware. Finally, detecting anomalies in network traffic often requires the use of time-consuming computational techniques, e.g. data mining and statistical analysis, and it is difficult for them to operate in real-time. Therefore, it is necessary to distribute the computational load required using distributed environments, i.e., parallel machines, cloud computing, etc.

Recently, for the reasons described above, the interest in distributed IDS has grown and different methodologies have been proposed. In the following subsections, we will firstly direct our attention to Collaborative Intrusion Detection Systems (CIDS). Basically, in the CIDS approach, the aim is to correlate information on attacks coming from different subnetworks. Next, we will describe solutions based on data mining techniques that, although not all of them are set up and tested in a fully parallel environment, nonetheless the solutions and the algorithms employed are particularly suitable for a parallel implementation. Lastly, we will include a brief discussion on solutions whose implementation takes full advantage of modern readily available parallel hardware. Namely we will review works, which implement their algorithms on GPGPU capable hardware, clusters, etc.

3.1. Collaborative IDS

The approach to intrusion detection based on the collaborative paradigm has the potential to design IDSs capable of detecting intrusions that occur across large networks and to correlate alarms coming from different sensors. Moreover, CIDSs reduce computational costs by sharing intrusion detection resources among different networks and they mitigate the problem of false alarms that would be generated by a single IDS operating alone. The main components of a CIDS, in accordance to the work of [109], are a *detection unit* and a *correlation unit*. The detection unit consists of multiple sensors deployed in a network that generates low level intrusion alerts. These low level intrusion alerts are combined by the correlation unit, then high level reports are generated and finally the real nature of the attacks is confirmed. The design of a CIDS faces then two main challenges, namely *system architecture* and *alert correlation* methodologies. From the point of view of the architecture, we can distinguish the *Centralized approach*, in which the data are collected by different sources but analyzed and correlated by a single central unit. This approach suffers from various drawbacks; first, as most of the work is done by a single unit, serious problems of reliability are present, since the failure of the central unit causes the CIDS stops from properly working. Moreover, a large computational power is required in order to prevent the degradation of the performance of the CIDS. In the *hierarchical approach*, the correlation units are structured in a pyramidal way, with layers that perform increasing complex analysis on the data acquired by the detection units, but the final analysis is always performed by a central unit, namely the top of the pyramid. This approach frees the central unit from a part of the computational load, with respect to the centralized approach, but suffers from similar analogous problems to the ones mentioned above. Finally, there is a *fully distributed approach* in which the detection and the correlation work is spread among different units. This approach can be set up by a P2P network, for instance, but it is often hard to coordinate a large number of nodes in order to attain a suitable detection accuracy and to guarantee the scalability of the global system. Indeed, the mechanism implemented to share information should be efficient enough to hinder the speed at which menaces spread across a network; moreover, it should remain so as nodes are added to the system. Just to give a glimpse of how hard this problem is, in some extreme cases, as in the case of the SQL-slammer worm, it has been estimated that as the worm began spreading through the Internet it infected over 90% of the vulnerable hosts just in ten

minutes. In the following, we review the main collaborative IDS and a summary of their characteristics is reported in Table 3.

In [108], the authors focus their attention on the problem of alert correlation, with the purpose of improving the scalability without renouncing the accuracy of the system, in particular for DDoS attacks and worms. In order to accomplish the above task, the architecture of the system is based on two main ideas. First of all, to speed up the correlation of the raw alarms, exploiting the knowledge on the different types of attacks, the search is limited to specific “patterns”, where pattern means a subset of features. For instance, the features of the network flow, “source address”, “source port” and “destination port” should be sufficient to correlate raw alarms for a broad class of DDoS attacks. Moreover, in order to deal with the large number of raw alerts coming from the various IDSs of the network, the alert correlation system is implemented in a fully distributed form, based on a P2P protocol. Each member of the alert network is made up of a detection unit and a correlation unit. The detection unit is responsible for collecting raw alarms relative to the local traffic. These alerts are analyzed by the correlation unit that implements a multi-dimensional clustering algorithm. In particular, the traffic is preliminarily filtered according to a combinations of key features or patterns of different attacks previously described, the filtered traffic is clustered and the traffic patterns are identified as suspicious by building a pattern lattice. Afterwards, the local results of this analysis are exchanged among the peers of the network for further analysis and correlation. The proposed system is tested on two data sets obtained from the Dshield.org website, which comprises a large number of firewall and NDIS logs collected from various platforms all over the world. In particular, one of the datasets employed in the testing contains data collected during the outbreak of the SQL-Slammer worm. The system is evaluated by conducting a large scale experiment on the PlanetLab network of the Princeton University in comparison to a fully centralized approach using metrics as detection accuracy and message exchange rate. The experiments demonstrates that the fully distributed approach is more efficient than the centralized approach in terms of the time required to correlate the alerts. Moreover, the above-mentioned probabilistic approach is really efficient in the case of stealthy attack scenarios.

In [15], a method to correlate alerts collected by different IDSs is presented. The IDSs inspect the local traffic by analyzing the content of the payload of requests. The analysis is based on the technique of n -grams analogously to the ANAGRAM algorithm[99]. The detection stage in the ANAGRAM system is improved following the technique of STAND (Sanitization Tool for ANomaly Detection) [26, 27] by a sanitization phase of the training set; then, the technique introduced in [28] is employed to tune the sensor parameters automatically. Once the payload is processed, the models associated with the normal traffic and with the suspicious packets are stored by using Bloom filters [14], which constitute their signatures. It is worth noting that storing the models of traffic by using Bloom filter has also the positive effect of preserving the privacy of the users during the correlation phase in which signatures are shared across different sites. Cross-site correlation of the alerts is obtained by using the method described in [71], i.e. the Worminator system. After that signatures are shared across sites, correlation is performed locally, by comparing the unencoded local alerts with the Bloom filter representing the alerts coming from remote systems. The system is evaluated on the datasets collected from the web servers of three different locations: , and . During the testing period the prototype was able to detect a considerable number of application-specific attacks previously unknown as well as a wide range of well-known attacks without human intervention. Finally, the system shows a very low rate of false positives.

In [48], another CIDS is presented. Its architecture is based on different local nodes, in which, during the training stage, the local network traffic is modeled by using an ensemble of gaussian mixture models. In practice, the ensemble is built by a variation of the AdaBoost algorithm, adapted to a streaming environment.

Local parametric detection models are exchanged among the nodes and then they are combined on the local nodes in order to constitute a global model, by using a process based on particle swarm optimization and SVM to select the fusion function. Experimental tests, performed on the KDD'99 dataset, shows the effectiveness of the system in improving the detection accuracy, in spite of exchanging a relative small quantity of data among the nodes, i.e., the parameters that determine the gaussian mixture models.

It should be remarked that since only models of the traffic are shared and not network traffic itself, this guarantees the privacy of the various nodes involved.

An interesting point of view on the topic of correlating intrusion data coming from different nodes of a given subnetwork is proposed in [72]. The solution proposed relies on the concept of *conversation exchange model* introduced in [42]. This scheme has the scope of modelling the network traffic dynamic using aggregate traffic features and decision trees that encode, for each protocol, the way in which the packets are exchanged in the subnetwork considered. This process permits estimation of the distribution of the number of packets exchanged across the nodes of the subnetwork during nominal operations. At detection time, this distribution is monitored and techniques inspired by statistical mechanic and thermodynamics are used to detect anomalies in the distribution of the packets due to malicious activities. This approach is tested using different scenarios, i.e., different attacks on different segments of the network either occurring in different time slots or simultaneously. For each scenario, the system was able to detect the presence of attacks; however, it is worth remarking that the system detects deviations in the normal network dynamic. In order to detect the type of the attack or the nodes in which it has occurred, a deeper inspection at the level of packet content or of the network flow should be used, as analogous approaches described in this review.

In [80], the authors present a system to cluster malwares based on their behavior. Simple statistical features are extracted from the traffic generated by the malware, i.e., the length of the requests, the number of requests, etc. Then, the models of the malware behavior are clustered in a two stage procedure. In the first stage, the ([106]) clustering algorithm is used, and its results are further refined by a hierarchical clustering. During the clustering process, cluster stability is performed by using the Davies-Boulding index in order to assess whether the clusters obtained are compact and well separated. Lastly, the signatures are generated from the clustering stages by using the Token-Subsequences algorithm [75]. Basically, for each cluster, signatures are generated considering substrings that are common to all payloads in that cluster. In [79], the same authors introduce some tweaks in the architecture, which presents a more balanced computational load and a better scalability in comparison with the original method. Experiments are conducted on many datasets of malware activities collected from various sources and the system proves to be able to generate signatures for different types of malwares.

Author(s)	Year	Technique	Correlation	Architecture	Dataset(s)	Evaluation
Zhou et al. [108]	2009	Raw alarms aggregation	Clustering	Fully distributed	Internet Storm Center	PlanetLab network of the Princeton University
Boggs et al. [15]	2011	<i>n</i> -grams analogously to the ANAGRAM algorithm [99], models are encoded by Bloom filters	Exchanging models of traffic	Fully distributed	Real traffic from different sources	Real traffic from different sources
Hong et al. [48]	2014	Ensemble of Gaussian mixture models	Particle swarm optimization, SVM	Fully distributed	KDD'99	None
McEachen et al. [72]	2007	Aggregate features	Conversation Exchange Model	Centralized	M.I.T. Lincoln Lab 1999	Appositely set up subnetwork
Perdisci et al. [80, 79]	2010, 2013	Statistical techniques	Hierarchical Clustering: BIRCH ([106])	Centralized	Malware samples	None

Table 3: Summary table of papers reviewed, CIDS. The column Evaluation indicates whether the framework is also evaluated on a real network.

3.2. Data mining-based approaches

As there are not many data mining distributed implementation of intrusion detection systems, this section also includes some works based on data mining that are particularly suitable to be implemented in a parallel/distributed environment. A summary of the main works, reviewed on the following, is reported in Table 4.

The work in [101] describes a framework for the problem of adaptive intrusion detection over unlabeled HTTP traffic streams, which follows the autonomic paradigm proposed by [58]. Indeed, it is able to self-label the incoming data stream, to update continuously the detection model and moreover, it is capable of adapting the detection model after a change happens in the data stream. The IDS works by inspecting packets at the payload level by essentially employing a 1-grams technique. After the preprocessing phase, in an initial stage, the data stream is clustered by an affinity propagation algorithm that employs a message passing technique named WAP (weighted affinity propagation). It is worth noting that clustering algorithms based on affinity propagation are graph-based and require a high computational load; therefore, they are generally more adapted to off-line analysis. However, the system adopts a message updating mechanism, which is suitable to be implemented in a distributed approach and that would extend its use to an online system. After the clustering phase is completed, the system analyzes the size and the sparseness of each cluster, in order to identify anomalous items. When a change is detected in the traffic, the models are rebuilt by restarting the clustering process. Three parameters control the rebuilding phase, i.e., the number of suspicious items, the length of the time window and the number of suspicious items since the last clustering stage. If one of these three parameters exceeds a certain threshold, the clustering phase is restarted. Finally, the system is compared with other existing clustering solutions: the k -Nearest Neighbor algorithm, the Principal Component Analysis ([54]), a one class SVM method and a version of the system based on the Sequential Karhunen-Loeve ([68]) transform. Testing is conducted on the KDD'99 dataset and on two large real HTTP traffic streams and the proposed system outperforms the other methods.

The approach in [23] presents an IDS (named UNIDS, *Network unsupervised intrusion detection system*) based on the misuse detection paradigm. The solution proposed, based on an ensemble clustering method, is suitable to be easily implemented on parallel/distributed architectures. In addition, although the authors test it on data stream coming from a single source, they point out it is well capable of dealing with data collected from different sources. The algorithm works in three phases: preprocessing, clustering and outlier identification. In the pre-processing phase, network traffic is captured from a single source using consecutive time windows of prefixed length. Afterwards, the captured traffic is aggregated into flows, and from each flow, some numerical features, i.e., source and destination network prefixes, traffic per time slot, etc. are extracted and a multi-dimensional vector representing the flow is obtained. A change detection algorithm is applied to the resulting vector and permits to flag some flows in the time slot as anomalous, which are passed to clustering phase. Then, in the clustering phase, a sub-space clustering algorithm is applied to the data, in the following way. First, the flows are aggregated using either IPsrc or IPdst aggregation keys and each aggregated flow is described by a number of traffic attributes. Multidimensional data, obtained as described before, are projected on multiple 2-dimensional subspaces of the feature space and a density-based DBSCAN algorithm [39] is performed on each subspace by using different notions of similarity or distance (i.e., a simple Euclidean distance, but also more complex statistical-based similarities). Finally, the results of the different clustering applied in each subspace are combined by an *evidence accumulation clustering* method [43]; in particular, the algorithm of *Evidence Accumulation for Ranking Outliers (EA4RO)* is employed, better described in [23]. In the last phase, the top-ranked outlying flows are flagged as anomalies, using a simple thresholding detection approach.

Experimental results are conducted both on the well-known KDD'99 dataset and on traffic captured

from two real networks. UNIDS is compared to traditional misuse based NIDs and it is comparable to the traditional approaches in its ability to detect unknown attacks; however, it is suitable to parallel computation, which permits a drastic reduction in the overall analysis time of the system.

An unsupervised approach based on clustering is introduced in [67]. The system adopts an adaptive grid algorithm, named fpMAFIA, based on the CLIQUE clustering technique [2], and its improvement pMAFIA described in [74]. The fpMAFIA algorithm basically employs *frequent-pattern tree* techniques in order to mine frequent item sets. This algorithm, based on the grid density clustering paradigm, starts by partitioning the features space in a grid, and part of the elaboration is basically performed by counting instances of the dataset in each element of the grid and establishing their relative density there, this involves the computation of the mean distance between two instances. Based on this elaboration grid elements are then merged to form clusters. The above analysis can then be implemented in parallel form by assigning, for instance, a certain number of grid elements to a single elaboration unit, e.g. a CPU core. The methodology proposed is tested on the KDD'99 dataset and it shows good performance in terms of elaboration speed and detection rate, but false positive rate is relatively high with respect to other methods proposed in the literature.

In [40], Folino et al. propose an ensemble-based distributed data mining algorithm, which adopts a genetic programming approach to build the classifiers composing the ensemble. The framework is used to improve the detection accuracy when classifying malicious or unauthorized network activity and the dataset is distributed across multiple nodes and the framework uses network profiles to improve the detection accuracy for the minority classes (i.e. the attacks). Experimental results show that the system outperforms the C4.5 algorithm and its boosting and bagging version as for the minority classes on the well-known KDD'99 dataset. The same authors [41] extend the above-cited approach by using ensembles specialized to detect particular types of attack and by adopting a non-trainable function to combine each specialized ensemble. The experiments are conducted on the recent ISCX dataset and demonstrate that the approach is able to cope with the intrusion detection task, even in the case of really unbalanced classes, by exploiting the advantages of the specialized classifiers.

Huang et al. [49] propose an architecture based on an ensemble of on line sequential extreme learning machines, which is a type of neural network that supports on line sequential learning, i.e., the predictor is updated as soon as new instances become available, see [69] for more details. The approach combines different ensemble techniques such as Bagging, Subspace Partitioning and Cross Validation; in addition, in order to speed up the overall training phase, the data are distributed across the nodes of a cluster by means of the Hadoop framework. Experimental testing is conducted on the well-known KDD'99 dataset and the parallel implementation obtain a good scalability and a considerable reduction of the execution time and maintains the same accuracy as the sequential version of the approach.

Another distributed architecture based on neural networks is presented in Bukhtoyarov et al. [21]. In this system, a probabilistic approach (more detail on this technique are given in [20]) is used for the generation of a neural network on each node of a network. Then, the neural networks are combined in an ensemble, in which the combination function is evolved by employing genetic programming using the same technique presented in [20]. Afterwards, each node classifies the traffic independently from the other nodes and it inquires the ensemble only in the case in which it is not “confident” in its prediction. “Confidence” is implemented by a function based on the signal level at the output of a neuron corresponding to the class determined by the individual neural network classifier; if the value of this function is below a prefixed threshold, then the individual classifier needs to require the help of the ensemble. The system is evaluated on the KDD'99 dataset in term of Detection Accuracy and False Positive Rate against standard ensemble algorithms and in general it is capable of equal or better score than the competition with optimal choice of the above mentioned threshold for the confidence function.

The architecture in Brahmi et al. [18] combines misuse based and anomaly detection techniques with the multi agent paradigm. Indeed, the analysis of the traffic is subdivided among many agents and some of them are devoted to collecting and pre-processing the traffic; then, the traffic is filtered by the so-named misuse detection agent, which checks it by using some rules related to known attacks. Then, the filtered traffic is passed to the anomaly detection agent that analyze it by using a clustering algorithm that combines K -means and DBSCAN and identify normal and abnormal traffic. Finally, the result of this analysis is passed to the rule mining agent that, by using an association rule-based technique, builds new rules that will be used by the misuse detection agent. The system is implemented using the JADE (Java Agent DEvelopment Framework), and the agent platform and is suitable to be implemented in a distributed environment, however the experiments are conducted on a sequential version of the framework. Experimental testing is conducted on a real traffic dataset, in which the attacks are injected by the *Metasploit*⁶ tool. The low bandwidth consumption and the network latency prove that the system is capable to cope with an increasing number of attacks and volume of network traffic. Moreover, the system is able to outperform largely used IDS systems, as SNORT, in term of detection rate and false positive rate.

Author(s)	Year	Tecniche(s)	Dataset(s)	Preprocess- ing	Metric(s)	Implemen- tation
Wang et al. [101]	2014	HTTP packet payload analysis based on 1-grams, WAP (Weighted Affinity Propagation) clustering	KDD'99, real traffic	None	ROC Curve	S
Casas et al. [23]	2012	DBSCAN algorithm [39], clustering ensemble: Evidence Accumulation for Ranking Outliers (EA4RO) ([23])	KDD'99, real traffic	Random Projections	ROC Curve, Detection Accuracy	S
Leung et al. [67]	2005	Grid density clustering [74]	KDD'99	None	ROC Curve	S
Folino et al. [40]	2010	Genetic Programming, Ensemble of Decision Trees	KDD'99	None	Detection Rate, False Positive Rate, Detection Accuracy	D
Folino et al. [41]	2016	Genetic Programming, Meta-Ensemble of Decision Trees	KDD'99, real traffic	None	Detection Rate, False Positive Rate, Detection Accuracy	D
Huang et al. [49]	2016	Ensemble of on line sequential extreme learning machines	KDD'99	None	Detection Accuracy	D
Bukhtoyarov et al. [21]	2014	Genetic Programming, Neural Networks	KDD'99	None	Detection Accuracy, False Positive Rate	D
Brahmi et al. [18]	2011	Multi Agent System, DBSCAN clustering, association rule mining	Real traffic	None	Detection Rate and False Positive Rate	S

Table 4: Summary table of papers reviewed, data mining distributed (In the last column (Implementation): D indicates Distributed implementation, S indicates Suitable to distributed implementation)

3.3. High-performance implementations

Recently, efforts have been made in order to take advantage of modern parallel hardware in the analysis of network traffic for the purpose of intrusion detection [97]. In fact, in the last few years, massively parallel hardware is readily available in the form of off the shelf components, namely multi-core CPUs and graphic hardware capable of general purpose computations (GPGPUs).

Most of the works present in the literature try to boost the signature verification stage by using a parallel implementation. For instance, there are projects that expand existing solutions in order to take advantage of GPGPU hardware, i.e., that is based on the well-known open source solution . Others works adopt a signature-based solution from scratch, and concentrate their efforts on implementing a very efficient signature verification based on parallel pattern matching algorithms [52, 8]. This approach can be extended by distributing the signature verification among different nodes. In this case, a particular attention is devoted to the problem of synchronizing the elaboration between nodes, see [94, 52, 96, 84].

⁶<http://www.metasploit.com/>

An interesting work is presented in [7], in which GPGPU hardware is paired to a cluster. In particular, computational tasks relative to signature matching are performed by the GPGPUs. The cluster is employed to process and mine the data streams of the alarms, and to assess the most common vulnerabilities and threats, by exploiting the advantages of the framework.

In order to take full advantage of a parallel implementation, a key point is to guarantee the scalability and a good load balancing. To this aim, the system proposed in [107] adopts a solution called negative pattern matching, which splices the preprocessed network traffic into small segments suitable to be analyzed in parallel and to perform a fine grained load balancing of the computational load. This technique ensures that no key information is lost in the process. Moreover, the rule set is subdivided into independent subsets, which can be applied independently, providing another means to parallelize the pattern matching process. Finally, the pattern matching is performed by a technique based on the ternary content addressable memory method. Experimental results show that the above-mentioned techniques are able to improve remarkably pattern matching performance and scalability in many real-world scenarios.

The approach in [3] uses an anomaly-based technique, which adopts a parallel local outlier algorithm. The technique used comprises a heavy phase of computation of the mutual distances of the points in some regions of the dataset. Therefore, a parallel implementation that increases the speed of the algorithm can be set up, as the computation of the distances can be performed independently. It would be interesting to implement more sophisticated solutions on parallel hardware and to test the systems on traffic data captured and processed in real time. For instance, modern database systems such as Oracle Database 10g (see [22]) provide all the necessary instruments to easily implement an intrusion detection architecture, from the data preprocessing to the phase of clustering and analysis. The possibility of exploiting the potentialities of the Oracle grid computing infrastructure is really interesting. In fact, this infrastructure enables sharing the content of the databases, or in other words the network traffic to be analyzed using the power capacity of many computing nodes. Therefore, it is possible to distribute the computational load in a transparent way and to ensure the availability of flexible on-demand computing resources capable of guaranteeing scalability and reliability. This infrastructure could provide a good testbed to compare different solutions and techniques.

4. Discussion: Lesson Learned and Open Issues

As discussed in the introduction to the paper, a modern NIDS has to cope with the increased speed of the underlying network connections. Increased network connections naturally mean an increase in the number of alarms generated, especially in the case of signature-based NIDS that works at the packet level. As a consequence, there is an ever increasing need for solutions that do not overwhelm human operators with a number of unmanageable alarms; indeed, it would be desirable to implement a mechanism that aggregates elementary alarms and presents higher level information to the operators.

The increase in the volume of data to be processed makes it really hard for an NIDS to work in real-time using sequential hardware. Finally, a modern NIDS needs to be capable of dealing with new undocumented attacks, for which a signature is not available (0-day attacks).

In view of the above remarks, solutions based on algorithms that can be implemented in parallel/distributed environment should be preferred. It is well known that currently it is easy to deploy hardware solutions capable of executing massively parallel algorithms, by using multi-core CPUs and GPGPU hardware, using off the shelf components. In this review, ensemble-based framework were individuated to be particularly suitable to exploit easily the power of parallel hardware.

By analyzing the data mining algorithms present in the literature for the intrusion detection task, though generally the authors note that their solutions are suitable to be implemented in parallel environments, there

is usually a lack of experiments that test the proposed solutions on real parallel hardware and on real network traffic.

Preprocessing steps and feature selection and extraction are key points in designing an efficient NIDS and the other part of the algorithm could benefit from an opportune design of these phases. In the reviewed works, we remarked the different preprocessing technique adopted, usually based on statistical, data-mining and information theoretic methods.

More in general, the works reviewed can be subdivided into two types. The first type considers the systems that inspect the traffic at the level of the packets payload. They usually concentrate on textual-based protocols, i.e. whose content is represented in plain text, such as for instance . In these cases the analysis of the payload is based on the technique of n -grams. The second type made up of systems that model the traffic at the level of network flow. The two types of approach have their weaknesses and strengths. First of all, the most computational intensive is generally the one that inspects package payload. It turns out that solutions based on the network flow are more adequate to modern environments involving high speed traffic and a consequent higher volume of data [93]. On the other hand, packet inspection-based methods are particularly efficient in the detection of attacks that present anomalous requests, i.e., cross-site scripting. Network flow analysis is particularly useful when applied to complex attacks as the case of DDoS. For the reasons explained above, it should be interesting to try and implement a hybrid solution taking the best of the two approaches. It is worth noticing that in any case both the approaches require a high computational load, especially if the analysis stage is based on the anomaly detection paradigm.

Consequently, it would be desirable to set up an anomaly detection mechanism relying on traffic models that are based on aggregate features; usually, traffic is aggregated in flows and flow properties are used as features; moreover, from an higher level, it is important also to consider aggregations and statistics based on these features such as entropies of flow characteristics for individual source of IP addresses, number of flows with given characteristics, and so on. For instance, recall the approach [72] described in Section 3.1, in which aggregate features are employed in order to build up a model of traffic for an entire subnetwork. Then, after a first analysis of the aggregate features, an algorithm could analyze, at a deeper level, only the time windows in which the change detector individuates an anomaly. As this mechanism usually must work in real-time, a tradeoff between accuracy and speed should be reached.

In the collaborative IDS approach, the alarms are shared between different detection points. This is attained by sharing the raw alarms, i.e. those generated locally by each NIDS must be exchanged among different sites and in addition, a mechanism to collect and analyze them must be implemented. The main advantages of this methodology is the ability to detect complex and correlated activities such as malware diffusion, distributed DoS and scans that span entire subnetworks. Moreover, the sharing mechanism could enable exchange of informations among systems concerning attacks individuated in a node, but not currently observed on the other parts of the network. The current literature on the subject suggests that, in order to prevent bottlenecks, the collection and the analysis of the alarms should be implemented in a fully distributed way. In practice, each node should be able to collect, analyze and elaborate the alarms and then to share globally the final results. Typically, this approach uses P2P network with their relative drawbacks; in fact, particular attention must be paid to the sharing mechanism in order to guarantee scalability and performance.

Moreover, for the problem of managing new attacks, techniques based on anomaly detection and, in particular, on clustering algorithms are in principle suitable to operate in an environment that requires the analysis of data streams containing previously unseen anomalous content. Nonetheless, generally there is a lack of experimentation, in which the solutions are tested in a real-life environment; consequently, it is not always clear whether the solutions proposed are able to operate in real-time or in near real-time. It is

also desirable to consider distributed clustering frameworks employing a mechanism for sharing the results across multiple nodes in a similar way to the alert correlation process described above; indeed, this sharing mechanism should be implemented again in a fully distributed way in order to prevent bottlenecks.

Sharing knowledge across multiple nodes either in the form of alerts or as the result of clustering and analysis, poses problems concerning the privacy of the involved subjects and institutions; in order to mitigate this problem as much as possible, the sharing mechanism should be based on abstract models of traffic, high level alerts, and should avoid the exchange of sensitive information such as the packet content. Recall for instance [48], in which only the parameters of the abstract models of traffic are exchanged, and [15], in which signatures of different types of traffic, first encoded by Bloom filters for protecting users' privacy, are shared.

As a general consideration, it is really hard to compare the different methods, based on different implementations, proposed in the literature. A common testbed on which to perform extensive testing on different datasets or even better in real environments and in different scenarios, would be remarkably useful for the intrusion detection community.

5. Conclusions

This work mainly reviews data mining algorithms that present an approach to network intrusion detection suitable to take advantage of modern parallel/distributed and cloud environments. Therefore, the aspects of the general design of a network intrusion detection systems that play a key role in designing an efficient system are considered. With this aim in mind, solutions based on the ensemble paradigm are analyzed, as this kind of algorithm is easily implementable on distributed environments and presents many advantages in the intrusion detection field. Collaborative intrusion detection systems are particularly interesting as, by correlating information coming from different nodes, they permit the discovery of more complex attacks and generally present a lower number of false positive alarms. In fact, they share informations with the other nodes in a subnetwork, either in the form of correlated alarms or in the form of models for detecting the attacks and therefore they minimize the rate of false positive alarms and are able to detect complex attacks that spread across entire subnetworks.

In addition, in order to design a modern architecture capable of coping with the high amount of data involved in modern high speed networks, high-performance distributed frameworks must be also considered. Finally, an overlooked issue in the intrusion detection field is the lack of a common distributed testbed, which permits the different solutions to be deployed and compared in a real environment.

Acknowledgment

This work has been partially supported by MIUR-PON under project PON03PE_00032_2 within the framework of the Technological District on Cyber Security.

References

- [1] Abdulla Amin Aburomman and Mamun Bin Ibne Reaz. A novel SVM-kNN-PSO ensemble method for intrusion detection system. *Applied Soft Computing*, 38:360 – 372, 2016.
- [2] Rakesh Agrawal, Johannes Gehrke, Dimitrios Gunopulos, and Prabhakar Raghavan. Automatic subspace clustering of high dimensional data. *Data Mining and Knowledge Discovery*, 11(1):5–33, 2005.
- [3] Malak Alshawabkeh, Byunghyun Jang, and David Kaeli. Accelerating the Local Outlier Factor algorithm on a GPU for intrusion detection systems. In *Proceedings of the 3rd Workshop on General-Purpose Computation on Graphics Processing Units*, GPGPU-3, pages 104–110, New York, NY, USA, 2010. ACM.

- [4] Fatemeh Amiri, Mohammad Mahdi Rezaei Yousefi, Caro Lucas, Azadeh Shakery, and Nasser Yazdani. Mutual information-based feature selection for intrusion detection systems. *Journal of Network and Computer Applications*, 34(4):1184 – 1199, 2011.
- [5] Davide Ariu and Giorgio Giacinto. HMMPayl: an application of HMM to the analysis of the HTTP payload. In Tom Diethe, Nello Cristianini, and John Shawe-Taylor, editors, *Proceedings of the First Workshop on Applications of Pattern Analysis, WAPA 2010, Cumberland Lodge, Windsor, UK, September 1-3, 2010*, volume 11, pages 81–87, 2010.
- [6] Davide Ariu, Roberto Tronci, and Giorgio Giacinto. HMMPayl: An intrusion detection system based on Hidden Markov Models. *Computers & Security*, 30(4):221 – 241, 2011.
- [7] S. R. Bandre and J. N. Nandimath. Design consideration of network intrusion detection system using Hadoop and GPGPU. In *Pervasive Computing (ICPC), 2015 International Conference on*, pages 1–6, Jan 2015.
- [8] Xavier J. A. Bellekens, Christos Tachtatzis, Robert C. Atkinson, Craig Renfrew, and Tony Kirkham. A highly-efficient memory-compression scheme for GPU-accelerated intrusion detection systems. In *Proceedings of the 7th International Conference on Security of Information and Networks, SIN '14*, pages 302:302–302:309, New York, NY, USA, 2014. ACM.
- [9] M. Hussain Bhuyan, D. K. Bhattacharyya, and J. K. Kalita. Survey on incremental approaches for network anomaly detection. *International Journal of Communication Networks and Information Security (IJCNIS)*, 3(3), 2011.
- [10] M.H. Bhuyan, D.K. Bhattacharyya, and J.K. Kalita. Network anomaly detection: Methods, systems and tools. *Communications Surveys Tutorials, IEEE*, 16(1):303–336, First 2014.
- [11] Monowar H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita. NADO: Network anomaly detection using outlier approach. In *Proceedings of the 2011 International Conference on Communication, Computing & Security, ICCCS '11*, pages 531–536, New York, NY, USA, 2011. ACM.
- [12] Monowar H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita. Surveying Port Scans and their detection methodologies. *Comput. J.*, 54(10):1565–1581, October 2011.
- [13] Monowar H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita. An effective unsupervised network anomaly detection method. In *Proceedings of the International Conference on Advances in Computing, Communications and Informatics, ICACCI '12*, pages 533–539, New York, NY, USA, 2012. ACM.
- [14] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [15] Nathaniel Boggs, Sharath Hiremagalore, Angelos Stavrou, and Salvatore J. Stolfo. Cross-domain collaborative anomaly detection: So far yet so close. In Robin Sommer, Davide Balzarotti, and Gregor Maier, editors, *Recent Advances in Intrusion Detection*, volume 6961 of *Lecture Notes in Computer Science*, pages 142–160. Springer Berlin Heidelberg, 2011.
- [16] Ali Borji. Combining heterogeneous classifiers for network intrusion detection. In Iliano Cervesato, editor, *Advances in Computer Science - ASIAN 2007. Computer and Network Security*, volume 4846 of *Lecture Notes in Computer Science*, pages 254–260. Springer Berlin Heidelberg, 2007.
- [17] Debojit Boro, Bernard Nongpoh, and Dhruba K. Bhattacharyya. Anomaly based intrusion detection using meta ensemble classifier. In *Proceedings of the Fifth International Conference on Security of Information and Networks, SIN '12*, pages 143–147, New York, NY, USA, 2012. ACM.
- [18] Imen Brahmi, Sadok Ben Yahia, Hamed Aouadi, and Pascal Poncelet. Towards a multiagent-based distributed intrusion detection system using data mining approaches. In *Proceedings of the 7th International Conference on Agents and Data Mining Interaction, ADMI'11*, pages 173–194, Berlin, Heidelberg, 2012. Springer-Verlag.
- [19] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [20] V. V. Bukhtoyarov and O. E. Semenkina. Comprehensive evolutionary approach for neural network ensemble automatic design. In *Evolutionary Computation (CEC), 2010 IEEE Congress on*, pages 1–6, July 2010.
- [21] Vladimir Bukhtoyarov and Vadim Zhukov. Ensemble-distributed approach in classification problem solution for intrusion detection systems. In Emilio Corchado, José A. Lozano, Héctor Quintián, and Hujun Yin, editors, *Intelligent Data Engineering and Automated Learning, IDEAL 2014*, volume 8669 of *Lecture Notes in Computer Science*, pages 255–265. Springer International Publishing, 2014.
- [22] M. M. Campos and B. L. Milenova. Creation and deployment of data mining-based intrusion detection systems in Oracle Database 10g. In *Machine Learning and Applications, 2005. Proceedings. Fourth International Conference on*. IEEE Computer Society, Dec 2005.
- [23] Pedro Casas, Johan Mazel, and Philippe Owezarski. Unsupervised Network Intrusion Detection Systems: Detecting the unknown without knowledge. *Computer Communications*, 35(7):772 – 783, 2012.
- [24] I. Corona, D. Ariu, and G. Giacinto. HMM-Web: A framework for the detection of attacks against web applications. In *Communications, 2009. ICC '09. IEEE International Conference on*, pages 1–6, June 2009.
- [25] Igino Corona, Giorgio Giacinto, Claudio Mazzariello, Fabio Roli, and Carlo Sansone. Information fusion for computer security: State of the art and open issues. *Information Fusion*, 10(4):274 – 284, 2009. Special Issue on Information Fusion in Computer Security.

- [26] G. F. Cretu, A. Stavrou, M. E. Locasto, S. J. Stolfo, and A. D. Keromytis. Casting out demons: Sanitizing training data for anomaly sensors. In *Security and Privacy, 2008. SP 2008. IEEE Symposium on*, pages 81–95, May 2008.
- [27] Gabriela F. Cretu, Angelos Stavrou, Salvatore J. Stolfo, and Angelos D. Keromytis. Data sanitization: Improving the forensic utility of anomaly detection systems. In *Proceedings of the 3rd Workshop on on Hot Topics in System Dependability, HotDep'07*, Berkeley, CA, USA, 2007. USENIX Association.
- [28] Gabriela F. Cretu-Ciocarlie, Angelos Stavrou, Michael E. Locasto, and Salvatore J. Stolfo. Adaptive anomaly detection via self-calibration and dynamic updating. In *Recent Advances in Intrusion Detection*, pages 41–60. Springer, 2009.
- [29] Marc Damashek. Gauging similarity with n-grams: Language-independent categorization of text. *Science*, 267(5199):843–848, 1995.
- [30] Belur V. Dasarathy. Intrusion detection. *Information Fusion*, 4(4):243–245, 2003.
- [31] David L. Davies and Donald W. Bouldin. A cluster separation measure. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, PAMI-1(2):224–227, April 1979.
- [32] Jonathan J. Davis and Andrew J. Clark. Data preprocessing for anomaly based network intrusion detection: A review. *Computers & Security*, 30(6-7):353 – 375, 2011.
- [33] Inderjit S. Dhillon, Subramanyam Mallela, and Rahul Kumar. A divisive information theoretic feature clustering algorithm for text classification. *J. Mach. Learn. Res.*, 3:1265–1287, March 2003.
- [34] S. Dua and X. Du. *Data Mining and Machine Learning in Cybersecurity*. CRC Press, 2011.
- [35] R. O. Duda, P. E. Hart, and D. G. Stork. *Pattern classification*. Pattern Classification and Scene Analysis: Pattern Classification. Wiley, 2001.
- [36] J. C. Dunn. Well-separated clusters and optimal fuzzy partitions. *Journal of Cybernetics*, 4(1):95–104, 1974.
- [37] Reda M. Elbasiony, Elsayed A. Sallam, Tarek E. Eltobely, and Mahmoud M. Fahmy. A hybrid network intrusion detection framework based on random forests and weighted k-means. *Ain Shams Engineering Journal*, 4(4):753 – 762, 2013.
- [38] Eleazar Eskin, Andrew Arnold, Michael Prerau, Leonid Portnoy, and Salvatore J. Stolfo. A geometric framework for unsupervised anomaly detection. In Daniel Barabá and Sushil Jajodia, editors, *Applications of Data Mining in Computer Security*, volume 6 of *Advances in Information Security*, pages 77–101. Springer US, 2002.
- [39] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, Portland, Oregon, USA, volume 96, pages 226–231. AAAI Press, 1996.
- [40] Gianluigi Folino, Clara Pizzuti, and Giandomenico Spezzano. An ensemble-based evolutionary framework for coping with distributed intrusion detection. *Genetic Programming and Evolvable Machines*, 11(2):131–146, June 2010.
- [41] Gianluigi Folino, Pietro Sabatino, and Francesco S. Pisani. Combining ensemble of classifiers by using genetic programming for cyber security applications. In *Applications of Evolutionary Computation - 19th European Conference, EvoApplications 2016, Porto, Portugal, 30 March - 1 April, 2016, Proceedings*, Lecture Notes in Computer Science. Springer International Publishing, 2016.
- [42] David Ford. Application of thermodynamics to the reduction of data generated by a non-standard system. *arXiv preprint cond-mat/0402325*, 2004.
- [43] Ana L. N. Fred and Anil K. Jain. Combining multiple clusterings using evidence accumulation. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 27(6):835–850, 2005.
- [44] P. Garcia-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, 28(1-2):18 – 28, 2009.
- [45] Giorgio Giacinto, Roberto Perdisci, Mauro Del Rio, and Fabio Roli. Intrusion detection in computer networks by a modular ensemble of one-class classifiers. *Information Fusion*, 9(1):69 – 82, 2008. Special Issue on Applications of Ensemble Methods.
- [46] N. Hoque, Monowar H. Bhuyan, R. C. Baishya, D. K. Bhattacharyya, and J. K. Kalita. Network attacks: Taxonomy, tools and systems. *Journal of Network and Computer Applications*, 40(0):307 – 324, 2014.
- [47] Shi-Jinn Horng, Ming-Yang Su, Yuan-Hsin Chen, Tzong-Wann Kao, Rong-Jian Chen, Jui-Lin Lai, and Citra Dwi Perkasa. A novel intrusion detection system based on hierarchical clustering and support vector machines. *Expert Systems with Applications*, 38(1):306 – 313, 2011.
- [48] Weiming Hu, Jun Gao, Yanguo Wang, Ou Wu, and S. Maybank. Online Adaboost-based parameterized methods for dynamic distributed network intrusion detection. *Cybernetics, IEEE Transactions on*, 44(1):66–82, Jan 2014.
- [49] Shan Huang, Botao Wang, Junhao Qiu, Jitao Yao, Guoren Wang, and Ge Yu. Parallel ensemble of online sequential extreme learning machine based on MapReduce. *Neurocomputing*, 174, Part A:352 – 367, 2016.
- [50] Lawrence Hubert and James Schultz. Quadratic assignment as a general data analysis strategy. *British Journal of Mathematical and Statistical Psychology*, 29(2):190–241, 1976.
- [51] Kenneth L. Ingham and Hajime Inoue. Comparing anomaly detection techniques for HTTP. In Christopher Kruegel, Richard Lippmann, and Andrew Clark, editors, *Recent Advances in Intrusion Detection*, volume 4637 of *Lecture Notes in Computer*

- Science, pages 42–62. Springer Berlin Heidelberg, 2007.
- [52] Muhammad Asim Jamshed, Jihyung Lee, Sangwoo Moon, Insu Yun, Deokjin Kim, Sungryoul Lee, Yung Yi, and Kyoung Soo Park. Kargus: A highly-scalable software-based intrusion detection system. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security, CCS '12*, pages 317–328, New York, NY, USA, 2012. ACM.
- [53] J.-S. R. Jang. ANFIS: adaptive-network-based fuzzy inference system. *Systems, Man and Cybernetics, IEEE Transactions on*, 23(3):665–685, May 1993.
- [54] I. T. Jolliffe. *Principal Component Analysis*. Springer Series in Statistics. Springer, 2002.
- [55] Song Jungsuk, Hiroki Takakura, Yasuo Okabe, and Kwon Yongjin. Unsupervised anomaly detection based on clustering and multiple one-class SVM. *IEICE transactions on communications*, 92(6):1981–1990, 2009.
- [56] H. Günes Kayacik, A. Nur Zincir-Heywood, and Malcolm I Heywood. Selecting features for intrusion detection: A feature relevance analysis on KDD 99 intrusion detection datasets. In *Proceedings of the third annual conference on privacy, security and trust*, 2005.
- [57] R.A. Kemmerer and G. Vigna. Intrusion detection: a brief history and overview. *Computer*, 35(4):27–30, Apr 2002.
- [58] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, January 2003.
- [59] R. Khanna and Huaping Liu. Control theoretic approach to intrusion detection using a distributed Hidden Markov Model. *Wireless Communications, IEEE*, 15(4):24–33, Aug 2008.
- [60] Rahul Khanna and Huaping Liu. System approach to intrusion detection using Hidden Markov Model. In *Proceedings of the 2006 International Conference on Wireless Communications and Mobile Computing, IWCMC '06*, pages 349–354, New York, NY, USA, 2006. ACM.
- [61] S. Kullback and R. A. Leibler. On information and sufficiency. *Ann. Math. Statist.*, 22(1):79–86, 03 1951.
- [62] P. Arun Raj Kumar and S. Selvakumar. Distributed denial of service attack detection using an ensemble of neural classifier. *Computer Communications*, 34(11):1328 – 1341, 2011.
- [63] P. Arun Raj Kumar and S. Selvakumar. Detection of distributed denial of service attacks using an ensemble of adaptive and hybrid neuro-fuzzy systems. *Computer Communications*, 36(3):303 – 319, 2013.
- [64] Ludmila Kuncheva. *Combining pattern classifiers: methods and algorithms*. John Wiley & Sons, 2004.
- [65] Haiguang Lai, Shengwen Cai, Hao Huang, Junyuan Xie, and Hui Li. A parallel intrusion detection system for high-speed networks. In Markus Jakobsson, Moti Yung, and Jianying Zhou, editors, *Applied Cryptography and Network Security*, volume 3089 of *Lecture Notes in Computer Science*, pages 439–451. Springer Berlin Heidelberg, 2004.
- [66] Edda Leopold and Jörg Kindermann. Text Categorization with Support Vector Machines. How to Represent Texts in Input Space. *Machine Learning*, 46(1-3):423–444, 2002.
- [67] Kingsly Leung and Christopher Leckie. Unsupervised anomaly detection in network intrusion detection using clusters. In *Proceedings of the Twenty-eighth Australasian Conference on Computer Science - Volume 38, ACSC '05*, pages 333–342, Darlinghurst, Australia, Australia, 2005. Australian Computer Society, Inc.
- [68] A. Levey and M. Lindenbaum. Sequential Karhunen-Loeve basis extraction and its application to images. *Image Processing, IEEE Transactions on*, 9(8):1371–1374, Aug 2000.
- [69] Nan-Ying Liang, Guang-Bin Huang, Paramasivan Saratchandran, and Narasimhan Sundararajan. A fast and accurate online sequential learning algorithm for feedforward networks. *Neural Networks, IEEE Transactions on*, 17(6):1411–1423, 2006.
- [70] Nick Littlestone and Manfred K. Warmuth. The weighted majority algorithm. *Inf. Comput.*, 108(2):212–261, February 1994.
- [71] Michael E. Locasto, Janak J. Parekh, Angelos D. Keromytis, and Salvatore J. Stolfo. Towards collaborative security and p2p intrusion detection. In *Information Assurance Workshop, 2005. IAW'05. Proceedings from the Sixth Annual IEEE SMC*, pages 333–339. IEEE, 2005.
- [72] John C. McEachen and Cheng Wai Kah. An analysis of distributed sensor data aggregation for network intrusion detection. *Microprocessors and Microsystems*, 31(4):263 – 272, 2007. Special Issue with selected papers from the 11th IEEE Symposium on Computers and Communications (ISCC'06).
- [73] Srinivas Mukkamala and Andrew H Sung. Identifying significant features for network forensic analysis using artificial intelligent techniques. *International Journal of digital evidence*, 1(4):1–17, 2003.
- [74] H. S. Nagesh, S. Goil, and A. Choudhary. A scalable parallel subspace clustering algorithm for massive data sets. In *Parallel Processing, 2000. Proceedings. 2000 International Conference on*, pages 477–484. IEEE, 2000.
- [75] J. Newsome, B. Karp, and D. Song. Polygraph: automatically generating signatures for polymorphic worms. In *Security and Privacy, 2005 IEEE Symposium on*, pages 226–241, May 2005.
- [76] Huu Hoa Nguyen, Nouria Harbi, and Jérôme Darmont. An efficient local region and clustering-based ensemble system for intrusion detection. In *Proceedings of the 15th Symposium on International Database Engineering & Applications, IDEAS '11*, pages 185–191, New York, NY, USA, 2011. ACM.
- [77] Kanubhai Patel and Bharat V Buddhadev. Machine learning based research for network intrusion detection: A state-of-the-art. *International Journal of Information and Network Security (IJINS)*, 3(3), 2014.

- [78] Roberto Perdisci, Davide Ariu, Prahlad Fogla, Giorgio Giacinto, and Wenke Lee. McPAD: A multiple classifier system for accurate payload-based anomaly detection. *Computer Networks*, 53(6):864 – 881, 2009.
- [79] Roberto Perdisci, Davide Ariu, and Giorgio Giacinto. Scalable fine-grained behavioral clustering of http-based malware. *Computer Networks*, 57(2):487 – 500, 2013.
- [80] Roberto Perdisci, Wenke Lee, and Nick Feamster. Behavioral clustering of http-based malware and signature generation using malicious network traces. In *Proceedings of the 7th USENIX Conference on Networked Systems Design and Implementation*, NSDI'10, pages 26–26, Berkeley, CA, USA, 2010. USENIX Association.
- [81] Abhinav S Raut and Kavita R Singh. Anomaly based intrusion detection-a review. *Int. J. on Network Security*, 5, 2014.
- [82] M. Riedmiller and H. Braun. A direct adaptive method for faster backpropagation learning: the RPROP algorithm. In *Neural Networks, 1993., IEEE International Conference on*, pages 586–591 vol.1, 1993.
- [83] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20(0):53 – 65, 1987.
- [84] Lambert Schaelicke, Kyle Wheeler, and Curt Freeland. SPANIDS: A scalable network intrusion detection Loadbalancer. In *Proceedings of the second Conference on Computing Frontiers*, CF '05, pages 315–322, New York, NY, USA, 2005. ACM.
- [85] R. E. Schapire. The strength of weak learnability. *Machine Learning*, 5(2):197–227, 1990.
- [86] R. E. Schapire. Boosting a weak learning by majority. *Information and Computation*, 121(2):256–285, 1995.
- [87] Bernhard Schölkopf, John C. Platt, John C. Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Comput.*, 13(7):1443–1471, July 2001.
- [88] C. Scott and R. Nowak. A neyman-pearson approach to statistical learning. *Information Theory, IEEE Transactions on*, 51(11):3806–3819, Nov 2005.
- [89] Siva S. Sivatha Sindhu, S. Geetha, and A. Kannan. Decision tree based light weight intrusion detection using a wrapper approach. *Expert Systems with Applications*, 39(1):129 – 141, 2012.
- [90] Raman Singh, Harish Kumar, and R.K. Singla. An intrusion detection system using network traffic profiling and online sequential extreme learning machine. *Expert Systems with Applications*, 42(22):8609 – 8624, 2015.
- [91] Jungsuk Song, Hiroki Takakura, Yasuo Okabe, and Koji Nakao. Toward a more practical unsupervised anomaly detection system. *Information Sciences*, 231(0):4 – 14, 2013. Data Mining for Information Security.
- [92] Yingbo Song, Angelos D Keromytis, and Salvatore Stolfo. Spectrogram: A mixture-of-markov-chains model for anomaly detection in web traffic. In *Network and Distributed System Security Symposium 2009: February 8-11, 2009, San Diego, California: Proceedings*, pages 121–135. Internet Society, 2009.
- [93] A. Sperotto, G. Schaffrath, R. Sadre, C. Morariu, A. Pras, and B. Stiller. An overview of IP flow-based intrusion detection. *Communications Surveys Tutorials, IEEE*, 12(3):343–356, Third 2010.
- [94] Giorgos Vasiladias, Michalis Polychronakis, and Sotiris Ioannidis. MIDeA: A multi-parallel intrusion detection architecture. In *Proceedings of the 18th ACM Conference on Computer and Communications Security, CCS '11*, pages 297–308, New York, NY, USA, 2011. ACM.
- [95] Emmanouil Vasilomanolakis, Shankar Karuppayah, Max Mühlhäuser, and Mathias Fischer. Taxonomy and survey of collaborative intrusion detection. *ACM Comput. Surv.*, 47(4):55:1–55:33, May 2015.
- [96] L. Vokorokos, M. Ennert, M. Cajkovsky, and A. Turinska. A distributed network intrusion detection system architecture based on computer stations using GPGPU. In *Intelligent Engineering Systems (INES), 2013 IEEE 17th International Conference on*, pages 323–326, June 2013.
- [97] Liberios Vokorokos, Michal Ennert, Marek Čajkovský, and Ján Raduňský. A survey of parallel intrusion detection on graphical processors. *Central European Journal of Computer Science*, 4(4):222–230, 2014.
- [98] Ke Wang, Gabriela Cretu, and Salvatore J. Stolfo. Anomalous payload-based worm detection and signature generation. In Alfonso Valdes and Diego Zamboni, editors, *Recent Advances in Intrusion Detection*, volume 3858 of *Lecture Notes in Computer Science*, pages 227–246. Springer Berlin Heidelberg, 2006.
- [99] Ke Wang, Janak J Parekh, and Salvatore J Stolfo. Anagram: A content anomaly detector resistant to mimicry attack. In *Recent Advances in Intrusion Detection*, pages 226–248. Springer, 2006.
- [100] Ke Wang and Salvatore J. Stolfo. Anomalous payload-based network intrusion detection. In Erland Jonsson, Alfonso Valdes, and Magnus Almgren, editors, *Recent Advances in Intrusion Detection*, volume 3224 of *Lecture Notes in Computer Science*, pages 203–222. Springer Berlin Heidelberg, 2004.
- [101] Wei Wang, Thomas Guyet, René Quiniou, Marie-Odile Cordier, Florent Masegla, and Xiangliang Zhang. Autonomic intrusion detection: Adaptively detecting anomalies over unlabeled audit data streams in computer networks. *Knowledge-Based Systems*, 70(0):103 – 117, 2014.
- [102] I.H. Witten, E. Frank, and M.A. Hall. *Data Mining: Practical Machine Learning Tools and Techniques: Practical Machine Learning Tools and Techniques*. The Morgan Kaufmann Series in Data Management Systems. Elsevier Science, 2011.
- [103] Xuanli Lisa Xie and G. Beni. A validity measure for fuzzy clustering. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 13(8):841–847, Aug 1991.

- [104] Jiong Zhang and M. Zulkernine. Anomaly based network intrusion detection with unsupervised outlier detection. In *Communications, 2006. ICC '06. IEEE International Conference on*, volume 5, pages 2388–2393, June 2006.
- [105] Jiong Zhang, M. Zulkernine, and A. Haque. Random-forests-based network intrusion detection systems. *Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on*, 38(5):649–659, Sept 2008.
- [106] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. Birch: an efficient data clustering method for very large databases. In *ACM SIGMOD Record*, volume 25, pages 103–114. ACM, 1996.
- [107] Kai Zheng, Zhiping Cai, Xin Zhang, Zhijun Wang, and Baohua Yang. Algorithms to speedup pattern matching for network intrusion detection systems. *Computer Communications*, 62:47 – 58, 2015.
- [108] Chenfeng Vincent Zhou, Christopher Leckie, and Shanika Karunasekera. Decentralized multi-dimensional alert correlation for collaborative intrusion detection. *Journal of Network and Computer Applications*, 32(5):1106 – 1123, 2009.
- [109] Chenfeng Vincent Zhou, Christopher Leckie, and Shanika Karunasekera. A survey of coordinated attacks and collaborative intrusion detection. *Computers & Security*, 29(1):124 – 140, 2010.